if $\Gamma \models w \simeq w'$ [7, Theorem 3.5.6]. If $\Gamma = \{\mathrm{E}\}$ consists of a single law E, we write $\mathcal{M}_{X,\mathrm{E}}$, $\sim_{\mathrm{E}}$, $\varphi_{f,\mathrm{E}}$ for $\mathcal{M}_{X,\{\mathrm{E}\}}$, $\sim_{\{\mathrm{E}\}}$, $\varphi_{f,\{\mathrm{E}\}}$ respectively.

In general, the free magma $\mathcal{M}_{X,\Gamma}$ is difficult to describe in a tractable form, but for some theories, one has a simple description. We give two simple examples here:

**Example 2.1** (Commutative and associative free magma)**.** The free magma $\mathcal{M}_{X,\{\mathrm{E43},\mathrm{E4512}\}}$ for the commutative law E43 and the associative law E4512 is the free abelian semigroup generated by $X$ (with $\iota_{X,\{\mathrm{E43},\mathrm{E4512}\}}$ the obvious embedding map).

**Example 2.2** (Left-absorptive free magma)**.** The free magma $\mathcal{M}_{X,\{\mathrm{E4}\}}$ for the left-absorptive law E4 is the magma with carrier $X$ and operation $x \diamond y = x$ (with $\iota_{X,\mathrm{E4}}$ the identity).

Every magma $\mathcal{M}$ has an opposite $\mathcal{M}^{\mathrm{op}}$, which has the same carrier but the opposite operation $x \diamond^{\mathrm{op}} y \coloneqq y \diamond x$. A magma $\mathcal{M}$ satisfies an equational law E if and only if its opposite $\mathcal{M}^{\mathrm{op}}$ satisfies the dual law $\mathrm{E}^*$, defined by reversing all the operations. For instance, the dual of $\mathrm{x} \diamond \mathrm{y} \simeq \mathrm{x} \diamond (\mathrm{y} \diamond \mathrm{z})$ (E327) is $\mathrm{y} \diamond \mathrm{x} \simeq (\mathrm{z} \diamond \mathrm{y}) \diamond \mathrm{x}$, which in our numbering system we rewrite in normal form as $\mathrm{x} \diamond \mathrm{y} \simeq (\mathrm{z} \diamond \mathrm{x}) \diamond \mathrm{y}$ (E395).

We then see that the implication graph has a duality symmetry: given two equational laws $\mathrm{E}_1, \mathrm{E}_2$, we have $\mathrm{E}_1 \models \mathrm{E}_2$ if and only if $\mathrm{E}_1^* \models \mathrm{E}_2^*$.

## 3. Formal foundations

All proofs in the ETP were ultimately formalized in the proof assistant language *Lean*, though in many cases the proofs were first written in an informal human document, which was then incorporated into the human-readable *blueprint* [42] that accompanied the formalization. Many of the computer-assisted proofs were also first generated as computer output from a source other than *Lean*, such as an ATP, and later converted to a *Lean* proof by a separate program custom-written for this task.

The project relied on *Lean*'s extensive *Mathlib* library, for instance to provide support for algebraic concepts such as the free group that arose in some of the more difficult constructions. Additional extensions to *Lean*, such as *duper* or *egg*, were employed by some participants in external forks of the repository, but we did not incorporate them into the master repository to simplify the version control process. As a consequence, some manual translation of proofs produced using such extensions to a proof that avoided such extensions were needed at various stages of the project.

The concept of a magma could be modeled by existing *Mathlib* classes such as `Mul`; however we chose early in the project to define a custom magma class `Magma` instead, as for some magma constructions the magma operation (which we denoted $\diamond$) was distinct from an existing multiplication structure $*$ on the same carrier. Most components of the *Lean* codebase were placed in namespaces to avoid collisions with each other, and with *Mathlib*.

Equational laws in the project were implemented both syntactically — as a structure `LawX` containing two words in a free group — as well as semantically, as a predicate `EquationX` that could be applied to a magma. Here `X` is the number assigned to the law. The semantic

```
@[equational_result]
theorem _root_.Equation1437_not_implies_Equation4269 :
    ∃ (G : Type) (_ : Magma G), Equation1437 G ∧ ¬ Equation4269 G := by
  use ℕ × Fin 3, ⟨op⟩
  constructor
  • intro x y z
    simp [op, add_assoc]
  • simp only [not_forall, op]
    use (0, 0), (2, 0)
    decide
```

FIGURE 5. A sample proof of a formalized implication, in this case that E1437 $\not\models$ E4269.

```
@[equational_result]
theorem «Facts from All4x4Tables [[1,2,3,4,5,0],[4,1,2,5,0,3],[3,0,5,2,1,4],
[0,5,4,3,2,1],[5,4,1,0,3,2],[2,3,0,1,4,5]]» :
  ∃ (G : Type) (_ : Magma G) (_ : Finite G), Facts G [1316, 2863] [411, 680,
  817, 1020, 1426, 2035, 2441, 2644, 2853, 2855, 2865, 2872, 2947, 3050,
  3253, 3456, 4270, 4283, 4290, 4380, 4598, 4605, 4656] :=
⟨Fin 6, «All4x4Tables [[1,2,3,4,5,0],[4,1,2,5,0,3],[3,0,5,2,1,4],[0,5,4,3,2,1],
[5,4,1,0,3,2],[2,3,0,1,4,5]]», Finite.of_fintype _, by decideFin!⟩
```

FIGURE 6. A computer generated `Facts` theorem, using an explicit finite magma of order 6 to refute several implications at once.

formulation (`EquationX`) was more convenient for proving or refuting specific implications, while the syntactic formulation (`LawX`) was preferred for implementing metatheorems, such as the use of duality between laws. *Lean*'s metaprogramming features proved to be vital to relate the two representations. A custom command, `equation`, was created for specifying equational laws. Elaborating the `equation` command generated both `EquationX` and `LawX` definitions from this description, as well as theorems relating them to each other. A similar construction was used to generate dual laws, where the dual law was given explicitly for simplicity.

To facilitate the automatic generation of an implication graph from the *Lean* codebase, a custom `@[equational_result]` tag was formed to attach to propositions in *Lean* to indicate that they were proving or refuting one or more implications; see Figure 5. A `conjecture` keyword was also created for implications or refutations which we wished to identify as having an informal proof that had yet to be formalized in *Lean*.

A single construction of a magma could satisfy multiple laws $E_1, E_2, \ldots$ and not satisfy others $E'_1, E'_2, \ldots$, leading to a large number of refutations of the form $E_i \not\models E'_j$. A custom `Facts` command was designed to organize such information efficiently; see Figure 6.

As an additional precaution against "exploit"-based proofs (such as those that might be contributed by an AI tool) *lean4checker* was used to ensure that no axioms were used in

*Lean* outside of a small trusted set. In particular, *Lean* tactics such as `native_decide` that relied on external tools were not permitted into the codebase.

Explicitly formalizing all 22 028 942 implications as theorems would lead to an infeasible compilation time in *Lean*. Instead, a reduced generating set of 10 657 positive implications and 586 925 negative implications were formalized, with the latter in turn mostly organized into a smaller number of `Facts` theorems as discussed above. The extension of these results to the rest of the implication graph via transitivity and duality is currently done by programs external to *Lean*, although in principle one could create an "end-to-end theorem" which completely establishes the implication graph within *Lean*.

Some lemmas generated in the project were suitable for upstreaming back to *Mathlib*, as well as several technical improvements to the *LeanBlueprint* software.

## 4. Project management

This project is, among other things, an experiment on how to organise large scale collaborations for mathematical work. In this section, we describe several aspects of the organisation of the collaborative effort.

4.1. **Problems of scale in mathematical collaboration.** In order to understand the scaling issues that can arise in large scale collaborations, it helps to revisit the mechanics of traditional mathematical collaborations and their limitations. While every collaboration is unique, there are some general patterns. A small number of contributors, usually under ten, who may know each other, join forces to tackle some class of problems. Typically the collaborators are almost all academics who share substantial amounts of common knowledge. They discuss the problem at hand together, typically with some shared written medium such as a whiteboard. After several rounds of discussion and refinement, different members of the collaboration come up with different pieces of a solution. These pieces are then put together via discussion and merging of write-ups over several iterations. Once the collaborators are reasonably confident about the correctness of their work, including theorem statements and proofs, they submit the paper for peer review. Thus the correctness of mathematical research relies on this basic cycle of discuss, solve, write, cross-check, and revise, followed by peer review. Ultimately the authors take responsibility for the contents of their research article. This joint responsibility for authorship is formally enshrined by mathematical societies. For instance, see point 4 of the EMS code of practice for joint responsibility [64].

However, this project involved over fifty contributors spread across the world with diverse academic and professional backgrounds. They collaborated across several timezones and countries over the internet. The aforementioned process does not scale. Collaborators do not usually know each other nearly as well as they would in a traditional project. Thus such a collaboration does not have the same level of mutual trust. Further, as the number of contributors grows beyond the single digits, it becomes increasingly difficult to ensure the robustness of each other's results, because of the sheer volume of material produced. Even delegating responsibility for the various pieces of mathematical work and integrating them into a coherent whole becomes difficult. Concretely, the scaling challenge manifests in several ways: